

Aurora vs RDS MySQL Cost Analysis

Scenario

Region: us-east-1 (N. Virginia)

Configuration:

- Instance Class: db.r6g.large
- Storage: 100 GB
- High Availability: Multi-AZ
- Database Engine: MySQL

Estimated Monthly Cost

Service	Estimated Monthly Cost
RDS MySQL (Multi-AZ)	\$377.05/month
Aurora MySQL	\$240.48/month

Actual pricing varies based on storage type, I/O usage, backup retention, data transfer, and Reserved Instance/Savings Plan discounts.

Key Observation

Contrary to the common assumption that Aurora is always more expensive, the AWS Pricing Calculator shows Aurora MySQL costing significantly less than RDS MySQL Multi-AZ for this specific workload.

Aurora is approximately **36% cheaper** in this comparison while still providing advanced capabilities such as automatic storage scaling, faster failover, built-in multi-AZ storage replication, reader endpoints, Aurora Global Database, and Aurora Serverless v2 support.

Why Aurora Is Cheaper in This Scenario

The primary reason is the difference in architecture.

RDS MySQL Multi-AZ maintains a separate standby instance in another Availability Zone, which increases infrastructure costs. Aurora, however, uses a distributed storage architecture

that automatically maintains six copies of data across three Availability Zones. This allows Aurora to provide high availability without relying on the same traditional standby model.

As a result, Aurora can sometimes deliver both better resilience and lower cost, depending on the deployment configuration.

When RDS MySQL Makes Sense

Choose RDS MySQL when:

- You need standard managed MySQL with minimal architectural complexity.
- Your application has modest performance requirements.
- You do not require Aurora-specific features such as Global Database or Serverless v2.
- Existing workloads are already optimized for RDS and there is little business value in migrating.

When Aurora Makes Sense

Choose Aurora when:

- High availability is a critical business requirement.
- Fast failover is important.
- You expect significant growth in traffic.
- You need read scaling through Aurora Replicas.
- You plan to deploy globally.
- You want automatic storage growth and modern scaling capabilities.

My Analysis

Based on this pricing exercise, Aurora MySQL appears to be the stronger option for this workload. It not only provides additional enterprise-grade features but also comes at a lower monthly cost than RDS MySQL Multi-AZ.

This exercise highlights an important lesson: database selection should not be based on assumptions about pricing. The AWS Pricing Calculator should always be used to evaluate real-world costs because the most feature-rich service is not necessarily the most expensive.

For this db.r6g.large production deployment, Aurora MySQL offers a compelling combination of lower cost, better scalability, and enhanced availability, making it the preferred choice.

AWS Database Decision Framework

Choose Amazon RDS when:

Amazon Web Services Amazon RDS

Use RDS when:

- You need a traditional relational database (MySQL, PostgreSQL, MariaDB, SQL Server, Oracle)
- Your workload is steady, predictable, and not extremely high scale
- You want a simpler, cost-effective managed database without advanced distributed architecture

Choose Amazon Aurora when:

Amazon Aurora

- You need high performance (up to ~5x MySQL / ~3x PostgreSQL performance)
- Your application requires high availability with automatic multi-AZ replication
- You expect scaling reads heavily (read replicas, global database support for DR and multi-region apps)

Choose Aurora Serverless when:

Amazon Aurora Serverless

- Your workload is intermittent or unpredictable (spiky traffic, dev/test environments)
- You want automatic scaling with no manual capacity planning

Real Example

OluPay fintech processes 500K transactions/day with strict SLA.

Recommended Choice: Amazon Aurora MySQL

Reason:

- Handles high transaction throughput reliably
- Provides fast failover and high availability
- Supports global database replication for disaster recovery
- Better suited for financial-grade latency and consistency requirements